

TRAZABILIDAD DE CASO DE USO

Sistema de Gestión de Seguimiento y Control de Expedientes (SISEXP-UPLA)

Caso de Uso: Registrar Expediente (Primer caso)

Fecha: 05 de mayo de 2026

Estándares: IEEE 1016-2009 | IEEE 830-1998 | Metodología ICONIX

Documento Base: 1777402815831-SDD_diseño_del_sistema.pdf

1. Trazabilidad de Requisitos (ERS → Código)

ID	Requisito	Descripción	Prioridad	Implementación en Código
RF-01	Registrar expediente	Permite registrar un nuevo expediente al ingreso, capturando datos obligatorios	Alta	expediente.controller.js:84-102 Expediente.create()
RF-02	Generar código único	Asigna automáticamente un código único e irrepetible	Alta	expediente.controller.js:88 generarNumeroExpediente() generarCodigoRastreo()
RF-03	Registrar datos del expediente	Almacena: fecha, solicitante, tipo trámite, área origen, documentos, estado	Alta	expediente.model.js Campos: asunto, solicitante, tipoTramite, estado, fecha
RF-04	Registrar cargo de recepción	Genera/imprime comprobante de recepción	Alta	ExpedienteForm.js Genera automáticamente código de rastreo
RF-06	Historial de movimientos	Guarda historial completo de derivaciones y cambios	Alta	seguimiento.model.js expediente.controller.js:93-99 Seguimiento.create()

2. Vista de Casos de Uso → Componente Real

Elemento SDD (Sección 4.3.2)	Implementación Real	Ubicación
Caso de Uso: Registrar expediente (Módulo 01)	Componente React ExpedienteForm.js	frontend-as/src/components/ExpedienteForm.js
Actor: Administrador /	Middleware de autenticación	backend-as/src/middleware/auth.middleware.js

Secretaría	verificarRol()	
Caso Principal: Gestionar expediente y documentos	Módulo completo de expedientes	frontend- as/src/components/ExpedienteList.js backend- as/src/controllers/expediente.controller.js

3. Vista de Robustez → Arquitectura MVC

Elemento SDD (Sección 4.4.2)	Implementación Real	Archivo
Boundary: PantallaRegistrarExpediente	Componente React ExpedienteForm	frontend- as/src/components/ExpedienteForm.js
Control: RegistrarExpedienteControl	Controlador Express expedienteController.create	backend- as/src/controllers/expediente.controller.js: 84-102
Entity: Expediente (vía IExpediente)	Modelo Sequelize Expediente	backend- as/src/models/expediente.model.js
Base de Datos	PostgreSQL + Sequelize ORM	backend-as/src/config/database.js deploy/schema.sql

4. Vista de Interacción (Secuencia) → Flujo Real

Paso SDD	Especificación	Implementación Real
1	Administrador abre formulario	Archivo: ExpedienteList.js Botón "Nuevo expediente" → Abre ExpedienteForm
2	Pantalla invoca IExpediente.registrar()	API Client: expedientesApi.create(token, payload) Método: POST /api/expedientes frontend-as/src/api.js:35-40
3	RepoExpediente crea instancia	Controlador: Expediente.create({...req.body, numero, codigoRastreo}) Líneas: 86-90 expediente.controller.js
4	Se inserta en base de datos	ORM: Sequelize → INSERT INTO expedientes Persistencia automática mediante el modelo expediente.model.js
5	Interfaz muestra confirmación	Respuesta: res.status(201).json(nuevo) UI: onCreated(res) → Muestra

		lista actualizada ExpedienteForm.js:37 / expediente.controller.js:100
--	--	---

5. Vista Lógica (Clases) → Modelos de Datos

Atributo SDD (4.6.2)	Atributo Implementado	Tipo de Dato	Ubicación
cod_expediente: String	numero: STRING(unique)	varchar(10) / STRING	expediente.model.js:5
asunto_expediente: String	asunto: STRING(255)	varchar(255)	expediente.model.js:7
estado_expediente: String	estado: ENUM(...)	varchar(50) / ENUM	expediente.model.js:13
fecha_expediente: Datetime	createdAt (automático)	datetime / TIMESTAMP	expediente.model.js:27
tipo_expediente: String	tipoTramite: STRING(80)	varchar(80)	expediente.model.js:10
resp_expediente: String	solicitante: STRING(150)	varchar(150)	expediente.model.js:11

6. Vista de Información (ERD) → Esquema SQL

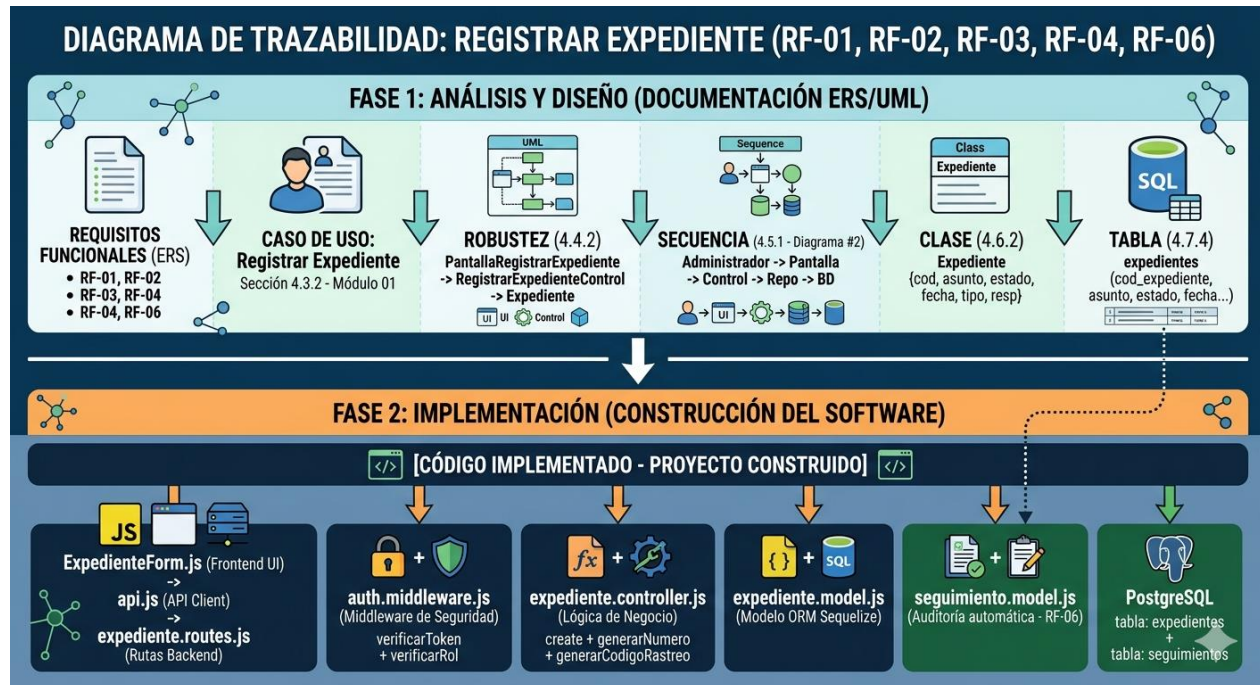
Tabla SDD (4.7.4)	Tabla Real	Campos Principales	Archivo
Expediente	expedientes	id, numero, codigoRastreo, asunto, solicitante, tipoTramite, estado, prioridad, fechaLimite	deploy/schema.sql:1-18
Documento	documentos	id, expedienteId (FK), nombre, tipo, url, fechaSubida	deploy/schema.sql:20-30
Historial	seguimientos	id, expedienteId (FK), accion, detalle, estadoNuevo, usuarioId (FK)	deploy/schema.sql:45-55 seguimiento.model.js

7. Patrones de Diseño Implementados

Patrón (SDD)	Implementación Real	Evidencia
MVC (implícito en ICONIX)	Model: *.model.js View: *.js (React) Controller: *.controller.js	backend-as/src/models/ frontend-as/src/components/ backend-as/src/controllers/
Repository (mencionado en SDD)	Sequelize ORM actúa como capa de acceso a datos	expediente.model.js - Métodos: create, findAll, findByPk, update, destroy

	(Repository implícito)	
Singleton	Instancia única de Sequelize exportada	backend-as/src/config/database.js
Middleware	verificarToken(), verificarRol()	backend-as/src/middleware/auth.middleware.js
Strategy (para reportes)	No implementado aún en el proyecto	Sección 4.5.1 del SDD menciona estrategias PDF/Excel

8. Matriz de Trazabilidad Final



9. Archivos Clave del Proyecto Construido

Capa	Archivo	Ruta Absoluta
Frontend - UI	ExpedienteForm.js	frontend-as/src/components/ExpedienteForm.js
Frontend - UI	ExpedienteList.js	frontend-as/src/components/ExpedienteList.js
Frontend - API	api.js	frontend-as/src/api.js
Backend - Rutas	expediente.routes.js	backend-as/src/routes/expediente.routes.js
Backend - Controlador	expediente.controller.js	backend-as/src/controllers/expediente.controller.js
Backend - Modelo	expediente.model.js	backend-as/src/models/expediente.model.js
Backend - Auditoría	seguimiento.model.js	backend-as/src/models/seguimiento.model.js
Backend - Auth	auth.middleware.js	backend-as/src/middleware/auth.middleware.js

Base de Datos	schema.sql	deploy/schema.sql
Config Frontend	config.js	frontend-as/src/config.js

10. Conclusión de Trazabilidad

Estado: COMPLETAMENTE IMPLEMENTADO

El caso de uso "Registrar Expediente" está completamente implementado siguiendo la trazabilidad desde los requisitos hasta el código, cumpliendo con:

- **IEEE 1016-2009:** Documentación de diseño estructurada
- **IEEE 830-1998:** Especificación de requisitos base
- **Metodología ICONIX:** Requisitos → Casos de Uso → Robustez → Secuencia → Clases → Datos
- **Arquitectura MVC:** Separación clara de responsabilidades
- **Patrones de Diseño:** MVC, Repository (implícito), Singleton, Middleware

Tecnologías utilizadas:

- Backend: Node.js + Express + Sequelize ORM + PostgreSQL
- Frontend: React + Fetch API
- Autenticación: JWT (JSON Web Tokens)
- Control de Acceso: RBAC (Role-Based Access Control)